

Task 1:

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Program using wait()

Code:

```
rohita@LaxmiDurga: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid;  
    int status;  
  
    printf("Parent process PID: %d\n", getpid());  
  
    for (int i = 1; i <= 3; i++)  
    {  
        pid = fork();  
  
        if (pid < 0)  
        {  
            perror("fork failed");  
            exit(1);  
        }  
  
        if (pid == 0)  
        {  
            printf("Child %d: PID = %d, Parent PID = %d\n",  
                i, getpid(), getppid());  
  
            sleep(i);  
            printf("Child %d completed.\n", i);  
            exit(0);  
        }  
    }  
  
    // Parent waits for any child  
    for (int i = 0; i < 3; i++)  
    {  
        pid = wait(&status);  
  
        if (WIFEXITED(status))  
        {  
            printf("Parent: Child PID %d terminated with status %d\n",  
                pid, WEXITSTATUS(status));  
        }  
    }  
  
    printf("Parent process completed.\n");  
  
    return 0;  
}
```

Output:

```
rohita@LaxmiDurga:~$ nano week4.c
rohita@LaxmiDurga:~$ gcc week4.c
rohita@LaxmiDurga:~$ ./a.out
Parent process PID: 1048
Child 1: PID = 1049, Parent PID = 1048
Child 2: PID = 1050, Parent PID = 1048
Child 3: PID = 1051, Parent PID = 1048
Child 1 completed.
Parent: Child PID 1049 terminated with status 0
Child 2 completed.
Parent: Child PID 1050 terminated with status 0
Child 3 completed.
Parent: Child PID 1051 terminated with status 0
Parent process completed.
rohita@LaxmiDurga:~$
```

Observation: • The parent creates **3 child processes**.

- `wait()` makes the parent wait for **any one child** to finish.
- The order in which children are collected depends on **which child terminates first**.
- The parent cannot select a particular child using `wait()`.
- After all children finish, the parent terminates.

Program using `waitpid()`

Code:

```
rohita@LaxmiDurga: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid[3];  
  
    printf("Parent process PID: %d\n", getpid());  
  
    for (int i = 0; i < 3; i++)  
    {  
        pid[i] = fork();  
  
        if (pid[i] < 0)  
        {  
            perror("fork failed");  
            exit(1);  
        }  
  
        if (pid[i] == 0)  
        {  
            printf("Child %d: PID = %d\n", i + 1, getpid());  
  
            sleep(3 - i);  
            printf("Child %d completed.\n", i + 1);  
  
            exit(10 + i);  
        }  
    }  
  
    // Parent waits for specific children  
    for (int i = 0; i < 3; i++)  
    {  
        int status;  
  
        waitpid(pid[i], &status, 0);  
  
        if (WIFEXITED(status))  
        {  
            printf("Parent: Child PID %d terminated with status %d\n",  
                pid[i], WEXITSTATUS(status));  
        }  
    }  
  
    printf("Parent process completed.\n");  
  
    return 0;  
}
```

Output:

```
rohita@LaxmiDurga:~$ gcc wek41.c  
rohita@LaxmiDurga:~$ ./a.out  
Parent process PID: 1150  
Child 1: PID = 1151  
Child 2: PID = 1152  
Child 3: PID = 1153  
Child 3 completed.  
Child 2 completed.  
Child 1 completed.  
Parent: Child PID 1151 terminated with status 10  
Parent: Child PID 1152 terminated with status 11  
Parent: Child PID 1153 terminated with status 12  
Parent process completed.
```

Observation:

`waitpid()` waits for a **specific child process** and provides better control than `wait()`. It helps the parent **synchronize and collect the child's exit status**.

Task 2:

Create a zombie process, observe it using the process table, then modify the program to eliminate the zombie using synchronization.

Code:

```
rohita@LaxmiDurga: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid;  
    int status;  
  
    pid = fork();  
  
    if (pid < 0)  
    {  
        perror("fork failed");  
        exit(1);  
    }  
  
    if (pid == 0)  
    {  
        printf("Child process: PID = %d\n", getpid());  
        printf("Child is terminating...\n");  
        exit(0);  
    }  
    else  
    {  
        printf("Parent process: PID = %d\n", getpid());  
        printf("Child PID = %d\n", pid);  
  
        // Parent waits for the specific child  
        waitpid(pid, &status, 0);  
  
        if (WIFEXITED(status))  
        {  
            printf("Parent: Child terminated normally.\n");  
            printf("Exit status: %d\n", WEXITSTATUS(status));  
        }  
  
        printf("Zombie process eliminated.\n");  
    }  
  
    return 0;  
}
```

Output:

```
-bash: ./zombie: No such file or directory
rohita@LaxmiDurga:~$ nano week42.c
rohita@LaxmiDurga:~$ gcc week42.c -o week42
rohita@LaxmiDurga:~$ ./week42
Parent process: PID = 1288
Child PID = 1289
Child process: PID = 1289
Child is terminating...
Parent: Child terminated normally.
Exit status: 0
Zombie process eliminated.
rohita@LaxmiDurga:~$
```

Observation:

- The child terminates, but remains in the **process table as a zombie** because the parent does not call `wait()`.
- Using `waitpid()` allows the parent to **collect the child's exit status and eliminate the zombie**.